

```
board = [' ' for _ in range(9)]
```

```
def display_board():
```

```
    print("\n")
```

```
    print(board[0], "|", board[1], "|", board[2])
```

```
    print("--+---+--")
```

```
    print(board[3], "|", board[4], "|", board[5])
```

```
    print("--+---+--")
```

```
    print(board[6], "|", board[7], "|", board[8])
```

```
    print("\n")
```

```
def check_winner(player):
```

```
    win_positions = [
```

```
        [0,1,2], [3,4,5], [6,7,8],
```

```
        [0,3,6], [1,4,7], [2,5,8],
```

```
        [0,4,8], [2,4,6]
```

```
    ]
```

```
    for pos in win_positions:
```

```
        if board[pos[0]] == board[pos[1]] == board[pos[2]] == player:
```

```
            return True
```

```
    return False
```

```
def check_draw():
```

```
    return ' ' not in board
```

```
def human_move():
```

```
    while True:
```

```
        pos = int(input("Enter position (1-9): ")) - 1
```

```
        if 0 <= pos <= 8:
```

```
    if board[pos] == ' ':
        board[pos] = 'X'
        break
    else:
        print("Position already occupied!")
else:
    print("Invalid position!")

def computer_move():
    print("Computer is making move...")

    for i in range(9):
        if board[i] == ' ':
            board[i] = 'O'
            break

print("TIC-TAC-TOE GAME")
print("Human Player = X")
print("Computer = O")

display_board()

while True:

    human_move()
    display_board()

    if check_winner('X'):
        print("Human Player Wins!")
        break
```

```

if check_draw():
    print("Game Draw!")
    break

computer_move()

display_board()

if check_winner('O'):
    print("Computer Wins!")
    break

if check_draw():
    print("Game Draw!")
    break

```

222222

```

board = [' ' for _ in range(9)]

def display_board():
    print("\n")
    print(board[0], "|", board[1], "|", board[2])
    print("---+---+---")
    print(board[3], "|", board[4], "|", board[5])
    print("---+---+---")
    print(board[6], "|", board[7], "|", board[8])
    print("\n")

def check_winner(player):

    win_positions = [
        [0,1,2], [3,4,5], [6,7,8],

```

```
[0,3,6], [1,4,7], [2,5,8],  
[0,4,8], [2,4,6]  
]
```

```
for pos in win_positions:
```

```
    if board[pos[0]] == board[pos[1]] == board[pos[2]] == player:
```

```
        return True
```

```
return False
```

```
def check_draw():
```

```
    return ' ' not in board
```

```
def human_move():
```

```
    while True:
```

```
        pos = int(input("Enter position (1-9): ")) - 1
```

```
        if 0 <= pos <= 8:
```

```
            if board[pos] == ' ':
```

```
                board[pos] = 'X'
```

```
                break
```

```
            else:
```

```
                print("Position already occupied!")
```

```
        else:
```

```
            print("Invalid position!")
```

```
def minimax(is_maximizing):
```

```
    if check_winner('O'):
```

```
        return 1
```

```
    if check_winner('X'):
```

```
        return -1
```

```
if check_draw():
```

```
    return 0
```

```
if is_maximizing:
```

```
    best_score = -100
```

```
    for i in range(9):
```

```
        if board[i] == '':
```

```
            board[i] = 'O'
```

```
            score = minimax(False)
```

```
            board[i] = ''
```

```
            if score > best_score:
```

```
                best_score = score
```

```
    return best_score
```

```
else:
```

```
    best_score = 100
```

```
    for i in range(9):
```

```
        if board[i] == '':
```

```
            board[i] = 'X'
```

```
            score = minimax(True)
```

```
            board[i] = ''
```

```
            if score < best_score:
```

```
        best_score = score

    return best_score

def computer_move():

    best_score = -100
    best_move = 0

    for i in range(9):

        if board[i] == '':

            board[i] = 'O'

            score = minimax(False)

            board[i] = ''

            if score > best_score:

                best_score = score
                best_move = i

    board[best_move] = 'O'

print("TIC-TAC-TOE USING MINI-MAX")
print("Human Player = X")
print("Computer = O")

display_board()

while True:

    human_move()
    display_board()

    if check_winner('X'):

        print("Human Player Wins!")
```

```
break
```

```
if check_draw():
```

```
    print("Game Draw!")
```

```
    break
```

```
print("Computer is making move...")
```

```
computer_move()
```

```
display_board()
```

```
if check_winner('O'):
```

```
    print("Computer Wins!")
```

```
    break
```

```
if check_draw():
```

```
    print("Game Draw!")
```

```
    break
```

33333

```
visited = set()
```

```
def dfs(state):
```

```
    x, y = state
```

```
    if y == 2:
```

```
        print("Goal State Reached:", state)
```

```
        return True
```

```
    if state in visited:
```

```
        return False
```

```
    visited.add(state)
```

```
    print("Current State:", state)
```

```

next_states = []

next_states.append((4, y))

next_states.append((x, 3))

next_states.append((0, y))

next_states.append((x, 0))

d = min(x, 3 - y)
next_states.append((x - d, y + d))

d = min(y, 4 - x)
next_states.append((x + d, y - d))

for new_state in next_states:
    if dfs(new_state):
        return True

return False

start = (0, 0)

print("DFS Traversal:")
dfs(start)

```

444

```

from collections import deque

```

```

visited = set()

```

```

def bfs(start):

```



```
queue = deque([start])

while queue:

    state = queue.popleft()

    x, y = state

    if y == 2:
        print("Goal State Reached:", state)
        return

    if state in visited:
        continue

    visited.add(state)

    print("Current State:", state)

    next_states = []

    next_states.append((4, y))

    next_states.append((x, 3))

    next_states.append((0, y))

    next_states.append((x, 0))

    d = min(x, 3 - y)
    next_states.append((x - d, y + d))

    d = min(y, 4 - x)
    next_states.append((x + d, y - d))
```

```
    for new_state in next_states:
        if new_state not in visited:
            queue.append(new_state)
```

```
start = (0, 0)
```

```
print("BFS Traversal:")
```

```
bfs(start)
```

5555

```
import heapq
```

```
goal_state = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 0]
]
```

```
def heuristic(state):
```

```
    distance = 0
```

```
    for i in range(3):
```

```
        for j in range(3):
```

```
            value = state[i][j]
```

```
            if value != 0:
```

```
                goal_x = (value - 1) // 3
```

```
                goal_y = (value - 1) % 3
```

```
            distance += abs(i - goal_x) + abs(j - goal_y)
```

```
    return distance
```

```

def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j

def generate_moves(state):
    x, y = find_blank(state)

    directions = [(-1,0), (1,0), (0,-1), (0,1)]

    moves = []

    for dx, dy in directions:
        nx, ny = x + dx, y + dy

        if 0 <= nx < 3 and 0 <= ny < 3:
            new_state = [row[:] for row in state]

            new_state[x][y], new_state[nx][ny] = new_state[nx][ny], new_state[x][y]

            moves.append(new_state)

    return moves

def a_star(start):
    visited = set()

    pq = []

    heapq.heappush(pq, (heuristic(start), 0, start))

    while pq:
        f, g, current = heapq.heappop(pq)

        state_tuple = tuple(map(tuple, current))

```

```

    if state_tuple in visited:
        continue

    visited.add(state_tuple)

    print("Current State:")
    for row in current:
        print(row)

    print()

    if current == goal_state:
        print("Goal State Reached")
        return

    for move in generate_moves(current):

        move_tuple = tuple(map(tuple, move))

        if move_tuple not in visited:
            h = heuristic(move)

            heapq.heappush(pq, (g + 1 + h, g + 1, move))

start_state = [
    [1, 2, 3],
    [4, 0, 6],
    [7, 5, 8]
]

a_star(start_state)

```

666

```
import heapq
```

```
goal_state = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 0]  
]
```

```
def heuristic(state):
```

```
    distance = 0
```

```
    for i in range(3):
```

```
        for j in range(3):
```

```
            value = state[i][j]
```

```
            if value != 0:
```

```
                goal_x = (value - 1) // 3
```

```
                goal_y = (value - 1) % 3
```

```
            distance += abs(i - goal_x) + abs(j - goal_y)
```

```
    return distance
```

```
def find_blank(state):
```

```
    for i in range(3):
```

```
        for j in range(3):
```

```
            if state[i][j] == 0:
```

```
                return i, j
```

```
def generate_moves(state):
```

```
    x, y = find_blank(state)
```

```
    directions = [(-1,0), (1,0), (0,-1), (0,1)]
```

```
    moves = []
```

```
    for dx, dy in directions:
```

```
        nx, ny = x + dx, y + dy
```

```

if 0 <= nx < 3 and 0 <= ny < 3:
    new_state = [row[:] for row in state]

    new_state[x][y], new_state[nx][ny] = new_state[nx][ny], new_state[x][y]

    moves.append(new_state)

return moves

def greedy_best_first_search(start):

    visited = set()

    pq = []

    heapq.heappush(pq, (heuristic(start), start))

    while pq:

        h, current = heapq.heappop(pq)

        state_tuple = tuple(map(tuple, current))

        if state_tuple in visited:
            continue

        visited.add(state_tuple)

        print("Current State:")
        for row in current:
            print(row)

        print()

        if current == goal_state:
            print("Goal State Reached")
            return

```

```

for move in generate_moves(current):

    move_tuple = tuple(map(tuple, move))

    if move_tuple not in visited:
        heapq.heappush(pq, (heuristic(move), move))

start_state = [
    [1, 2, 3],
    [4, 0, 6],
    [7, 5, 8]
]

greedy_best_first_search(start_state)

```

7777

```

import math

def print_board(board):
    for row in board:
        print(row)

def find_empty(board):
    n = len(board)

    for i in range(n):
        for j in range(n):
            if board[i][j] == 0:
                return (i, j)

    return None

def is_valid(board, row, col, num):
    n = len(board)

```

```
for j in range(n):
```

```
    if board[row][j] == num:
```

```
        return False
```

```
for i in range(n):
```

```
    if board[i][col] == num:
```

```
        return False
```

```
box_size = int(math.sqrt(n))
```

```
start_row = row - row % box_size
```

```
start_col = col - col % box_size
```

```
for i in range(start_row, start_row + box_size):
```

```
    for j in range(start_col, start_col + box_size):
```

```
        if board[i][j] == num:
```

```
            return False
```

```
return True
```

```
def solve_sudoku(board):
```

```
    empty = find_empty(board)
```

```
    if not empty:
```

```
        return True
```

```
    row, col = empty
```

```
    n = len(board)
```

```
    for num in range(1, n + 1):
```

```
        if is_valid(board, row, col, num):
```

```
            board[row][col] = num
```

```
            if solve_sudoku(board):
```

```
                return True
```



```
board[row][col] = 0
```

```
return False
```

```
board = [
```

```
[5, 3, 0, 0, 7, 0, 0, 0, 0],
```

```
[6, 0, 0, 1, 9, 5, 0, 0, 0],
```

```
[0, 9, 8, 0, 0, 0, 0, 6, 0],
```

```
[8, 0, 0, 0, 6, 0, 0, 0, 3],
```

```
[4, 0, 0, 8, 0, 3, 0, 0, 1],
```

```
[7, 0, 0, 0, 2, 0, 0, 0, 6],
```

```
[0, 6, 0, 0, 0, 0, 2, 8, 0],
```

```
[0, 0, 0, 4, 1, 9, 0, 0, 5],
```

```
[0, 0, 0, 0, 8, 0, 0, 7, 9]
```

```
]
```

```
if solve_sudoku(board):
```

```
    print("Solved Sudoku:")
```

```
    print_board(board)
```

```
else:
```

```
    print("No solution exists")p
```